Projet : Optimisation Par Colonie De Fourmi

Cours CSC_4OS02 - Année 2026

Júlia Ellen Dias Leite
Helena Guachalla de Andrade
Mateus Bastos Soares

Table des matières

1 Introduction **2**

2 Méthodologie **2**
2.1 Environnement 2
2.2 Instrumentalisation 2

3 Résultats Baseline **2**
3.1 Profiling Temporel (gprof) 2
3.2 Décomposition temporelle par itération 3

4 Vectorisation (SoA) **3**
4.1 Approche Structure of Arrays 3
4.2 Profiling de la version vectorisée 3
4.3 Analyse de Performance 4

5 Parallélisation OpenMP **4**
5.1 Stratégie de Parallélisation 4
5.2 Accélération en fonction du nombre de threads 5

6 Stratégie de Parallélisation 1 **5**
6.1 Implémentation MPI hybride 5
6.2 Synchronisation des phéromones 6
6.3 Mesures et granularité calcul/communication 6
6.4 Résultats expérimentaux MPI 6
6.5 Analyse de la Parallélisation 6
6.6 Calcul de l'Accélération (Speedup) 7

7 Stratégie de Parallélisation 2 **8**
7.1 Principe général 8
7.2 Gestion des fourmis 8
7.3 Synchronisation des cellules fantômes 9
7.4 Évaporation des phéromones 9

8 Analyse des Goulots d'Étranglement **9**

9 Conclusion **10**

1 Introduction

Ce rapport présente l'optimisation progressive d'un simulateur d'algorithme de colonie de fourmis (ACO) appliqué à la recherche de chemins optimaux sur un terrain fractal. Nous détaillons les étapes de vectorisation et parallélisation du code, en nous appuyant systématiquement sur des profils `gprof` pour guider les optimisations.

2 Méthodologie

2.1 Environnement

Pour garantir une comparaison équitable entre les versions, nous utilisons un scénario fixe :

- Graine pseudo-aléatoire : `2026`
- Taille du `grid` : 512×512 cellules
- Nombre de fourmis : $m = 5000$
- Paramètre de bruit : $\alpha = 0{,}7$
- Coefficient d'évaporation : $\beta = 0{,}999$
- Taux d'exploration : $\varepsilon = 0{,}8$
- Position du nid : $(256, 256)$
- Source de nourriture : $(500, 500)$

Les différents profils `gprof` sont obtenus en compilant avec l'option `PROFILE=yes`. En complément, chaque version est instrumentée pour mesurer explicitement les temps moyens par itération (temps de `advance_time`, évaporation, mise à jour des phéromones, rendu et temps total). Les comparaisons de performance entre versions sont donc faites principalement à partir de ces mesures internes normalisées par itération, plus robustes que le seul temps cumulé du `gprof`.

2.2 Instrumentalisation

Pour l'analyse de l'optimisation du code, nous mesurons séparément les temps d'exécution par itération de la fonction `advance_time`, de l'évaporation des phéromones, de la mise à jour de la carte de phéromones, du rendu graphique et du temps total d'une itération complète.

La fonction responsable du rendu graphique (`Renderer::display`) est également mesurée, et bien qu'une grande partie du temps de calcul lui soit destinée, la parallélisation ne la prendra pas en compte.

3 Résultats Baseline

3.1 Profiling Temporel (gprof)

Le premier profil `gprof` est obtenu sur la version originale (programmation orientée objet). Sur cette version, le benchmark instrumenté réalise **3 304** itérations en **60 s**, soit un temps moyen mesuré de **18,16 ms/itération**. Le *flat profile* `gprof` correspondant donne un temps total échantillonné de **17,06 s**.

Table 1 – Flat profile gprof — version originale (temps total : 17,06 s)

Fonction	**Temps (s)**	**%**	**Appels**	**ms/appel**
`ant::advance`	9,34	54,75	16 520 000	≈ 0,00057
`Renderer::display`	4,89	28,66	3 304	1,48
`advance_time`	2,58	15,12	3 304	0,78
`fractal_land::compute_subgrid`	0,04	0,23	87 376	0,00
Autre (init, main, allocations, ...)	0,21	1,24	–	–
Total	**17,06**	**100 %**		

Le nombre total d'appels à `ant::advance` est cohérent avec une moyenne de 5 000 appels par itération (16,52M appels / 3 304 itérations).

3.2 Décomposition temporelle par itération

Les mesures internes donnent pour la version originale : `advance_time` = 3,776 ms, évaporation = 0,707 ms, mise à jour des phéromones = 0,016 ms, rendu = 11,988 ms et temps total = 18,161 ms par itération. Le rendu graphique domine donc nettement le temps total mesuré, avec environ les deux tiers du temps d'une itération complète. Le profil `gprof` confirme cependant que, à l'intérieur du noyau de simulation, la quasi-totalité du coût de `advance_time` provient des appels à `ant::advance`, tandis que la génération du terrain fractal reste négligeable.

On constate également que la quasi-totalité du temps d'`advance_time` est passée dans les appels individuels à `ant::advance`, ce qui justifie de cibler en priorité cette fonction pour la vectorisation.

4 Vectorisation (SoA)

4.1 Approche Structure of Arrays

Au lieu de maintenir un objet par fourmi, nous utilisons des vecteurs :

- `pos_x[], pos_y[]` : positions
- `state[]` : états (chargée/non chargée)
- `seed[]` : graines PRNG

Cette transformation permet d'améliorer la localité mémoire et facilite l'utilisation de SIMD par le compilateur sur les boucles qui parcourent les fourmis.

4.2 Profiling de la version vectorisée

La version vectorisée est profilée avec un temps total `gprof` de **21,84 s**. Sur l'exécution instrumentée, elle réalise **4 800** itérations en **120 s**, soit **25,01 ms/itération**. Le tableau suivant rassemble les informations du profil.

Table 2 – Flat profile gprof — version vectorisée (temps total : 21,84 s)

Fonction	Temps (s)	%	Appels	ms/appel
`advance_time`	14,31	65,52	4 800	2,98
`Renderer::display`	6,88	31,50	4 800	1,43
`fractal_land::compute_subgrid`	0,03	0,14	87 376	0,00
Autre (init, main, allocations, ...)	0,62	2,84	–	–
Total	**21,84**	**100 %**		

On observe que l'appel élémentaire `ant::advance` ne figure plus dans le profil, car sa logique est désormais intégrée dans le noyau vectorisé `advance_time`, qui traite toutes les fourmis via des tableaux contigus.

4.3 Analyse de Performance

En comparant les temps par itération entre la version orientée objet et la version vectorisée, on obtient :

— Baseline : 18,16 ms/itération.

— Version vectorisée : 25,01 ms/itération.

Dans cette configuration de benchmark, le temps moyen par itération de la version vectorisée est donc sensiblement *plus élevé* que celui de la version orientée objet, ce qui correspond à un speedup global $S \approx 18{,}16/25{,}01 \approx 0{,}73$. Les mesures instrumentées montrent en particulier que le rendu graphique reste dominant (environ 20,08 ms sur 25,01 ms), ce qui masque une partie du gain obtenu sur le noyau de calcul lui-même.

Le noyau de simulation est désormais regroupé dans `advance_time`, qui concentre environ 66 % du temps `gprof`, confirmant ainsi la pertinence de le cibler avec des optimisations OpenMP ultérieures.

5 Parallélisation OpenMP

5.1 Stratégie de Parallélisation

La version OpenMP parallèle principalement la boucle externe sur les fourmis dans le noyau vectorisé, en s'appuyant sur `#pragma omp parallel for` :

```
// Boucle sur les fourmis
#pragma omp parallel for
for (int i = 0; i < num_ants; ++i) {
    // mise a jour position, etat...
}
```

Cette parallélisation est naturelle car chaque fourmi est traitée indépendamment au sein de la boucle principale. En revanche, l'évaporation et la mise à jour globale des phéromones restent séquentielles dans cette version, ce qui limite mécaniquement le speedup global accessible.

5.2 Accélération en fonction du nombre de threads

Pour quantifier l'impact d'OpenMP, nous avons mesuré le temps moyen par itération avec 2, 4 et 8 threads, en comparant au temps de la version vectorisée séquentielle (1 thread). Le temps de référence est ici celui de la version vectorisée, soit $T_1 =$ 25,01 ms/itération. Nous reportons également le temps de `advance_time`, plus représentatif du noyau effectivement parallélisé.

Table 3 – Résultats OpenMP à partir du benchmark instrumenté

Threads	**1**	**2**	**4**	**8**
Advance_time (ms)	3,04	2,72	2,32	1,33
Itération (ms)	25,01	25,12	25,20	20,44
Speedup noyau	1,00	1,12	1,31	2,28
Speedup global	1,00	1,00	0,99	1,22

Les profils `gprof` des versions OpenMP confirment que `advance_time` reste un poste important du calcul, mais les mesures instrumentées montrent que le gain sur ce noyau est partiellement masqué par le coût du rendu graphique. Entre 1 et 8 threads, le temps de `advance_time` est divisé par environ 2,28, tandis que le temps total d'itération ne s'améliore que d'un facteur 1,22. Cela indique que, dans cette configuration, le rendu graphique devient un facteur limitant majeur pour l'accélération globale.

6 Stratégie de Parallélisation 1

6.1 Implémentation MPI hybride

La version implemente une stratégie hybride combinant MPI et OpenMP. Chaque processus MPI maintient une copie complète de l'environnement, mais ne simule qu'un sous-ensemble des fourmis. À l'intérieur de chaque processus, la boucle `advance_time` reste parallélisée avec OpenMP, de sorte que le parallélisme est organisé sur deux niveaux : MPI répartit les fourmis entre processus, et OpenMP répartit les fourmis locales entre threads.

Le compteur de nourriture a été adapté au contexte distribué. Pendant chaque itération, chaque processus maintient un compteur local correspondant à la quantité de nourriture récoltée par ses fourmis. En fin d'itération, ces compteurs locaux sont agrégés avec `MPI_Allreduce`, de manière à ce que la valeur affichée par le rendu et enregistrée dans les journaux corresponde à la somme globale sur l'ensemble des processus.

Pour éviter la création de multiples fenêtres graphiques, l'interface SDL est restreinte au rang 0. Seul ce processus initialise SDL, crée la `Window` et le `Renderer`, traite les événements utilisateur et effectue les appels à `Renderer::display`. Les autres rangs continuent à exécuter le calcul (mise à jour des fourmis, évaporation, synchronisation des phéromones), mais sans affichage. Cette organisation permet de conserver un parallélisme efficace sans dupliquer l'interface graphique sur chaque processus MPI.

6.2 Synchronisation des phéromones

La synchronisation des phéromones entre processus est gérée par une fonction dédiée `sync_pheromones`. Pour l'implémenter, nous avons fourni un accès direct au tampon interne de la structure `pheronome`, ce qui permet de sérialiser et d'échanger efficacement la carte de phéromones. À chaque itération, chaque processus envoie sa carte locale au rang 0 via `MPI_Gatherv`. Le rang 0 combine ensuite ces cartes et rediffuse le résultat à tous les processus à l'aide de `MPI_Bcast`, de sorte que tous travaillent à nouveau sur un état global cohérent des phéromones à l'itération suivante.

6.3 Mesures et granularité calcul/communication

Le code de la version hybride a également été instrumenté pour fournir des mesures plus fines. À chaque itération, nous chronométrons séparément le temps passé dans `advance_time`, le temps d'évaporation, le temps de mise à jour des phéromones, le temps de rendu (sur le rang 0), le temps total de l'itération ainsi que le temps dédié aux communications MPI (gather, broadcast, réductions). À partir de ces mesures, nous dérivons des métriques simples de granularité, telles que le rapport calcul/communication et une estimation du volume de données synchronisées par appel, afin d'évaluer dans quelle mesure le coût de communication peut limiter le speedup lorsque le nombre de processus augmente.

6.4 Résultats expérimentaux MPI

Nous avons ensuite évalué la version MPI (stratégie de réplication du domaine). Les profils gprof montrent que le temps total est principalement réparti entre la fonction `advance_time`, la synchronisation des phéromones (`sync_pheromones`) et, lorsque le rendu est activé, `Renderer::display`.

Table 4 – Résultats MPI+OpenMP (OMP=1) : temps, speedup et ratio compute/comm

Processus	**1**	**2**	**4**	**8**
Temps (ms/it)	29,10	31,71	35,55	43,68
Speedup S_p	1,00	0,92	0,82	0,67
Comm/Total (% est.)	5	21	36	54
Total itérations	4125	3779	3371	2741

Les mesures montrent que, pour 4 threads, le coût de synchronisation des phéromones représente déjà plus de 40 % du temps total, ce qui limite fortement le speedup potentiel de la stratégie de réplication du domaine. À 8 threads, le temps total diminue (environ 3,29 ms/itération), mais la part relative de la communication reste significative, et la scalabilité reste inférieure à ce qu'on pourrait attendre d'un noyau largement parallélisable.

6.5 Analyse de la Parallélisation

Afin d'évaluer l'efficacité des différentes stratégies de parallélisation (OpenMP en mémoire partagée et MPI avec réplication du domaine), nous avons extrait les métriques clés des exécutions. Le tableau ci-dessous résume le temps de calcul pur des fourmis, le

temps de synchronisation de la carte des phéromones et le temps total par itération. Les données de rendu graphique sont exclues de l'analyse du speedup car l'affichage est limité au processus maître (Rang 0).

Table 5 – Synthèse des performances (OpenMP / MPI) en millisecondes par itération

MPI	**OMP**	**Calcul (ms)**	**Sync. (ms)**	**Itération (ms)**	**Ratio Cal/Comm**
Scalabilité OpenMP (Mémoire Partagée)					
1	2	3.07	1.52	29.22	2.01
1	4	2.99	1.56	29.33	1.91
1	8	3.05	1.64	29.63	1.85
Scalabilité MPI (Réplication du Domaine - Première façon)					
2	1	1.25	6.67	31.32	0.18
4	1	1.29	12.27	35.56	0.10
8	1	0.95	25.01	44.96	0.03
Exécution Hybride					
2	8	1.45	6.37	31.38	0.22
4	8	1.56	13.18	35.99	0.11

L'analyse de ces résultats met en évidence la limitation structurelle de la « Première façon » décrite dans le sujet :

— **Stagnation sous OpenMP (MPI=1)** : L'augmentation des threads de 2 à 8 n'apporte pas de gain sur le temps de calcul des fourmis ($\approx$ 3,0 ms). Cela indique une saturation précoce de la bande passante mémoire ou un surcoût de gestion des threads (*overhead*) qui efface les bénéfices de la parallélisation locale.

— **Goulot d'étranglement MPI (OMP=1)** : La répartition des fourmis entre plusieurs processus remplit son rôle initial : le temps de calcul individuel chute de 3,05 ms à 0,95 ms avec 8 processus. Cependant, le temps de synchronisation des phéromones explose, passant de 1,64 ms à 25,01 ms.

En conséquence, le ratio Calcul/Communication s'effondre brutalement à 0,03. La communication globale et répétitive de l'intégralité de la grille à chaque itération annihile complètement les gains obtenus sur le calcul, augmentant paradoxalement le temps total d'exécution au lieu de le réduire.

6.6 Calcul de l'Accélération (Speedup)

Afin d'isoler l'impact du surcoût de communication, nous calculons deux accélérations distinctes : l'accélération sur le noyau de calcul pur (S_{calcul}) et l'accélération globale de l'itération (S_{global}). En l'absence d'une exécution purement séquentielle dans cette série de mesures, nous prenons la configuration `MPI=1, OMP=2` comme référence de base ($S = 1{,}00$).

Table 6 – Évolution de l'accélération (Speedup) en fonction de la configuration

MPI	**OMP**	**Calcul (ms)**	$S_{\mathbf{calcul}}$	**Itération (ms)**	$S_{\mathbf{global}}$
		Référence et OpenMP			
1	2	3.07	1.00	29.22	1.00
1	4	2.99	1.03	29.33	1.00
1	8	3.05	1.01	29.63	0.99
		Stratégie MPI (Réplication du Domaine)			
2	1	1.25	2.46	31.32	0.93
4	1	1.29	2.38	35.56	0.82
8	1	0.95	3.23	44.96	0.65
		Exécution Hybride			
2	8	1.45	2.12	31.38	0.93
4	8	1.56	1.97	35.99	0.81

Ce tableau illustre le paradoxe de cette strategie : la distribution du calcul des fourmis est efficace. Par exemple, avec 8 processus MPI, le temps de calcul des fourmis est divisé par plus de trois ($S_{\text{calcul}} = 3{,}23$).

Cependant, le speedup global (S_{global}) chute à 0,65. Cela signifie que la simulation est en réalité plus lente avec 8 processus qu'avec un seul. L'accélération obtenue sur la charge de travail (les fourmis) est entièrement effacée par la complexité de l'opération `MPI_Allreduce` qui doit répliquer une carte complète sur chaque processus à chaque itération.

7 Stratégie de Parallélisation 2

7.1 Principe général

La seconde approche de parallélisation repose sur une décompostion de domaine. Contrairement à la première méthode où chaque processus possède une copie complète de l'environnement, ici la carte est divisée en sous-domaines adjacents, chacun étant attribué à un processus distinct. Chaque processus gère alors :

— La portion de carte qui lui est assignée (valeurs du terrain, types de cellules)
— Les tableaux de phéromones V_1 et V_2 pour son sous-domaine
— Les fourmis présentes sur son territoire

Chaque sous-domaine est étendu d'une couche de cellules fantômes (ou *ghost cells*) sur ses bordures. Ces cellules supplémentaires permettent de stocker une copie des valeurs des cellules voisines appartenant aux processus adjacents, nécessaire pour les calculs locaux qui requièrent la connaissance du voisinage.

7.2 Gestion des fourmis

Chaque processus maintient localement un tableau des fourmis présentes sur son sous-domaine, contenant pour chacune : sa position, son état (chargée/non chargée), et son identifiant unique ; et un compteur local d'unités de nourriture collectées.

La répartition initiale des fourmis dépend du mode d'initialisation choisi (toutes dans la fourmilière ou uniformément sur la grille). La fourmilière étant localisée sur un seul processus, si l'initialisation est centralisée, ce processus démarre avec l'ensemble des fourmis.

Lorsqu'une fourmi franchit la frontière d'un sous-domaine, elle doit être transférée au processus voisin. La procédure de migration s'effectuerait comme suit :

1. Après le déplacement des fourmis, chaque processus identifie celles dont la nouvelle position se trouve hors de son domaine
2. Ces fourmis sont regroupées par direction de sortie (nord, sud, est, ouest)
3. Des communications point-à-point sont effectuées avec les processus voisins pour échanger les fourmis sortantes contre les fourmis entrantes
4. Chaque processus intègre les fourmis reçues dans sa structure de données locale

Cette migration garantit que chaque fourmi est toujours gérée par le processus responsable de la cellule sur laquelle elle se trouve.

7.3 Synchronisation des cellules fantômes

Pour calculer correctement la mise à jour des phéromones sur une cellule s, une fourmi a besoin de connaître les valeurs V_1 et V_2 des quatre cellules voisines. Pour les cellules situées en bordure de sous-domaine, ces voisins appartiennent au processus adjacent. Un échange de bordures est donc nécessaire :

1. Après chaque mise à jour locale des phéromones, chaque processus envoie les valeurs de ses cellules de bordure aux processus voisins correspondants
2. Simultanément, il reçoit les valeurs des bordures de ses voisins et met à jour ses cellules fantômes
3. Ces communications peuvent être effectuées de manière non-bloquante pour recouvrir le temps de communication par du calcul

7.4 Évaporation des phéromones

L'évaporation des phéromones est une opération purement locale : chaque processus applique indépendamment le coefficient β à l'ensemble des cellules de son sous-domaine.

8 Analyse des Goulots d'Étranglement

Les profils successifs permettent d'identifier plusieurs goulots d'étranglement structurels :

— **Mémoire** : malgré le passage en SoA, les accès à la grille de phéromones restent non séquentiels et entraînent des défauts de cache, particulièrement pour les trajectoires de fourmis très dispersées.

— **Rendu graphique** : `Renderer::display` représente environ 33 % du temps dans la version vectorisée et plus de 35 % dans la baseline, et n'est pas facilement parallélisable au-delà de quelques optimisations internes (double-buffering, batching).

— **Overhead de fenêtre** : lors de petites exécutions, la création de la fenêtre peut dominer le profil (jusqu'à 60–90 % du temps sur certains runs courts), ce qui impose d'utiliser des runs longs pour des mesures représentatives.

— **Communication MPI** : dans les stratégies distribuées, la synchronisation des phéromones et l'échange de fourmis peuvent limiter le speedup si la fréquence de communication est trop élevée.
— **Équilibrage de charge** : la distribution des fourmis entre processus MPI est statique et uniforme en nombre de fourmis, mais le coût réel par fourmi dépend du terrain et des trajectoires suivies. Il peut donc subsister des déséquilibres en temps de calcul d'un rang à l'autre. Le même phénomène existe au niveau OpenMP, où la répartition par défaut du `parallel for` ne tient pas compte de cette variabilité de charge.

9 Conclusion

Ce projet a permis d'explorer l'optimisation d'une simulation de colonie de fourmis à travers différentes stratégies logicielles et matérielles. Guidée par le profilage des performances, la refonte de la structure de données en *Structure of Arrays*, combinée à une parallélisation en mémoire partagée via OpenMP, a rempli son objectif sur le noyau de calcul.

La première stratégie MPI, fondée sur la réplication complète de l'environnement sur chaque processus, est simple à mettre en œuvre et permet de répartir efficacement le calcul des fourmis. En revanche, elle devient rapidement limitée par la synchronisation globale de la carte des phéromones, dont le coût croît avec le nombre de processus. À l'inverse, la seconde stratégie, basée sur une décomposition spatiale du domaine, est plus complexe à programmer mais paraît plus adaptée à une montée en charge réelle, car elle remplace une communication globale coûteuse par des échanges locaux entre voisins.

Enfin, les performances restent fortement dépendantes de la machine utilisée : nombre de cœurs, caches, bande passante mémoire, coût des synchronisations OpenMP et efficacité des communications MPI. Sur une machine de bureau, ces contraintes matérielles peuvent rapidement limiter les gains attendus. En pratique, la piste la plus prometteuse pour aller plus loin serait donc de réduire l'impact du rendu lors des benchmarks et d'explorer la seconde stratégie de parallélisation, plus scalable à grande échelle.